

Uniformed Search: Search Problem and Examples Part-2

Introduction

In the pursuit of creating intelligent systems capable of solving a vast array of real-world challenges, a fundamental step involves formalizing how these problems are represented. This lecture delves into the core components required to define any problem as a search problem, a foundational concept in Artificial Intelligence. While the ultimate goal of modeling "every problem in the world" remains lofty, this framework provides a powerful and generalizable approach for a significant subset of problems. Understanding this formalization is crucial because it dictates how an AI agent perceives and interacts with its environment, laying the groundwork for subsequent algorithmic solutions. We will explore how a "domain designer" translates a real-world scenario into a structured problem that an AI can tackle, focusing on the implicit nature of the problem space rather than explicit enumeration.

Core Concepts

Term	Definition	Significance
State	A specific configuration or snapshot of the problem environment at a given moment.	Central to AI, it defines the current situation and is the basis for all agent actions and evaluations.
Domain Designer	The human expert responsible for defining the problem's states, actions, and goals for the AI agent.	Bridges the gap between real-world problems and the formal representation required by AI systems.
Atomic Representation	A state representation where each state is treated as an indivisible unit, often represented by a unique identifier.	Simplifies state comparison but hides internal structure, making it harder for AI to reason about state properties.
Goal Test	A function or "black box" that determines if a given state	Essential for identifying when a solution has been

Term	Definition	Significance
	satisfies the problem's objective.	found, allowing for multiple possible goal states.
Expand Function	A function that, given a state, returns all possible next states reachable by taking a single action.	Defines the transitions between states, implicitly constructing the problem's search space without explicit storage.
Start State	The initial configuration of the problem from which the AI agent begins its search.	The unique starting point for any problem instance, crucial for initiating the search process.
Search Problem	A formal definition comprising a start state, an expand function, and a goal test, often with associated action costs.	Provides a standardized framework for AI agents to find paths from an initial state to a goal state.
Uninformed Search	Search strategies that operate without any additional information about the "goodness" or proximity of states to the goal.	Represents the most basic level of search, relying solely on the problem's formal definition to explore the state space.
Search Tree	A conceptual tree structure formed by expanding states from the start state, where nodes represent states and edges represent actions.	Visualizes the exploration process, though often too large to be fully constructed in memory.
Search Node	A data structure used in search algorithms that contains a state, its parent node, the action taken to reach it, and its cost/depth.	Differentiates from a pure "state" by including meta-information vital for path reconstruction and search management.

Detailed Analysis

1. Formalizing the Search Problem with Black Boxes

The core idea behind formalizing a search problem for an AI agent is to provide it with a set of "black box" functions rather than an explicit, pre-computed graph of all possible states and transitions. This approach is critical because many real-world problems have state spaces too vast to be fully enumerated or stored in memory. The domain designer, a human expert, is responsible for crafting these black boxes.

The three fundamental components provided to the AI agent are:

- * **Start State:** This is the initial configuration from which the agent begins its search. It is typically unique for a given problem instance.

- * **Goal Test:** This function takes any given state as input and returns a boolean value (yes/no) indicating whether that state satisfies the problem's goal.

Importantly, there can be multiple goal states, so the term "a goal state" is often preferred over "the goal state."

- * **Expand Function (or Successor Function):** Given a current state, this function returns a list of all possible next states that can be reached in a single step, along with the actions (operators) required to transition to them. This function implicitly defines the graph structure of the problem.

This "black box" approach allows the AI system to explore the problem space dynamically, generating states only as needed, rather than requiring the entire problem graph upfront. This is a crucial distinction from traditional graph algorithms like Dijkstra's, which assume the entire graph is available.

2. Limitations of the Current Formulation

While powerful, this basic search problem formulation has inherent limitations that restrict the types of problems it can model:

- * **Determinism:** The current model assumes that taking an action in a given state always leads to a single, predictable next state. There is no notion of uncertainty or stochastic outcomes. If an action could lead to one of several possible states with certain probabilities, this formulation would not suffice.

- * **Single-Agent Environment:** The model implicitly assumes a single agent acting in the world, and that agent's actions are the only things changing the environment. It does not naturally account for adversarial environments where other agents might be making moves, or dynamic environments that change independently of the agent's actions.

These limitations highlight that while this framework is general for a significant class of problems, more advanced AI techniques are needed to address uncertainty and multi-agent interactions.

3. Example: The 8-Puzzle Problem

A classic example illustrating the search problem formulation is the 8-puzzle. This puzzle consists of eight numbered tiles and one blank space arranged in a 3x3 grid. The goal is to rearrange the tiles from an arbitrary starting configuration to a specific target configuration (e.g., 1-8 in order, with the blank in the bottom right).

- **States:** Each possible configuration of the tiles on the 3x3 board constitutes a state. For instance, 7, 2, 4, 5, gap, 6, 8, 3, 1 is one such state.
- **Actions (Operators):** The actions involve moving the blank space (or, equivalently, shifting a tile into the blank space). Possible actions are move gap right, move gap left, move gap up, and move gap down. At most, four actions are possible from any given state, and at least two (when the gap is in a corner).
- **Goal Test:** The goal test checks if the current board configuration matches the desired final configuration (e.g., 1, 2, 3, 4, 5, 6, 7, 8, gap). For the 8-puzzle, there is typically only one goal state.
- **Path Cost:** The cost of an action is usually 1 (each move counts as one step). The path cost is the total number of steps taken to reach the goal.

The challenge with the 8-puzzle, even for its small size, is the immense state space. For a 3x3 grid, there are 9! (9 factorial) possible arrangements, which is 362,880 states. While this might be enumerable, larger versions like the 4x4 (16!) or 5x5 (25!) puzzles have state spaces that are astronomically large, making explicit graph construction impossible. This reinforces the need for the implicit graph representation provided by the expand function.

4. Other Problem Examples

The search problem framework is versatile and can be applied to various domains:

* **Robotic Assembly:**

* **States:** Partial configurations of the assembled product, indicating which parts are connected.

* **Actions:** Adding a new part, or lower-level actions like moving robot joints to place a component.

* **Goal Test:** Checking if the final product assembly is complete and correct.

* **Shortest Path Problems (e.g., Navigation):**

* **States:** Cities or locations.

- * **Actions:** Driving between connected cities.
- * **Goal Test:** Being in the destination city (e.g., Bucharest from Arad).
- * **Path Cost:** Total time or distance traveled.
- * **N-Queens Problem:** Placing N queens on an N x N chessboard such that no two queens attack each other (horizontally, vertically, or diagonally).
- * **Start State:** An empty chessboard.
- * **States:** Partial configurations of the board with some queens placed, ensuring no current attacks.
- * **Actions:** Adding one more queen to the board. A clever approach might involve placing queens column by column.
- * **Goal Test:** All N queens are placed on the board, and no two queens are attacking each other.

These examples demonstrate the flexibility of defining states and actions to model diverse problems within this framework.

5. Search Space Exploration: Trees vs. Graphs

When an AI agent starts from the initial state and repeatedly applies the expand function, it generates a structure that can be conceptualized as a search space.

- * **Expanding States:** From the start state, the expand function reveals immediate successor states. Recursively applying this process builds out the search space.
- * **Search Tree:** If the agent does not keep track of previously visited states, it can revisit the same physical state multiple times through different action sequences. This leads to a search tree, where a single physical state might appear as multiple distinct nodes in the tree. For example, moving a gap up and then down in the 8-puzzle returns to the original state, creating a cycle that would be represented as distinct paths in a tree.
- * **Search Graph:** To avoid redundant exploration and potentially infinite loops, a search algorithm can perform "duplicate detection." This involves checking if a newly generated state has already been visited. If so, the algorithm can choose to ignore it or update its path if a shorter one is found. This results in a search graph. However, maintaining a record of all visited states for duplicate detection can consume significant memory, especially for large state spaces. For simplicity in initial studies, AI often begins with the search tree version, acknowledging its potential inefficiencies.

6. State vs. Search Node

It is crucial to distinguish between a "state" and a "search node" in the context of implementing search algorithms:

- * **State:** This refers to the abstract representation of the physical configuration of

the problem. It's the content that defines "where we are" in the problem space (e.g., the arrangement of tiles in the 8-puzzle).

* **Search Node:** This is a data structure used by the search algorithm to manage the exploration process. A search node contains a state but also includes additional meta-information vital for the algorithm:

* A pointer to its **parent node** (to reconstruct the path).

* The **action** taken to reach this state from its parent.

* The **cost** incurred to reach this state from the start state.

* The **depth** of the node in the search tree.

* Other information relevant to specific search strategies (e.g., heuristic values).

This distinction is important for practical implementation, as the search node encapsulates all the necessary information for the algorithm to operate effectively, beyond just the problem's current configuration.

Key Takeaways

- [] The domain designer defines the problem for the AI agent using a start state, a goal test, and an expand function.
- [] The expand function implicitly defines the problem's graph, which is often too large to be stored explicitly in memory.
- [] The basic search formulation assumes a deterministic, single-agent environment without uncertainty.
- [] "A goal state" is preferred over "the goal state" because problems can have multiple valid solutions.
- [] Problems like the 8-puzzle and N-Queens can be effectively modeled as search problems by defining their states, actions, and goal tests.
- [] Uninformed search problems lack additional information (heuristics) to guide the search towards the goal.
- [] The process of expanding states generates a search space, which can be viewed as a tree or a graph, with duplicate detection being key for graph representation.
- [] A "state" represents the physical configuration, while a "search node" is a data structure containing a state plus meta-information for the search algorithm.

Conclusion

This lecture has laid the theoretical groundwork for understanding how diverse problems can be formalized into a standard search problem representation for AI. By abstracting real-world scenarios into a start state, an expand function, and a goal

test, we empower AI agents to systematically explore potential solutions. This framework, while initially limited to deterministic, single-agent environments, serves as a robust foundation. The ability to model problems like the 8-puzzle, robotic assembly, or the N-Queens problem demonstrates the broad applicability of this approach. As we move forward, this understanding will be crucial for developing and analyzing various search algorithms, first in the context of uninformed search where no additional guidance is available, and later for more sophisticated informed search techniques that leverage problem-specific knowledge to find solutions more efficiently. The distinction between a conceptual "state" and an algorithmic "search node" also prepares us for the practical implementation challenges of these powerful AI tools.